Experiment 1: Customer, Branch, Account and Loan

Aim: To create tables with primary and foreign keys, insert records, and execute SQL queries.

1. Create tables

```
CREATE TABLE Customer (
    cust_id NUMBER PRIMARY KEY,
    cust_name VARCHAR2(50) NOT NULL,
    city VARCHAR2(30),
    phone VARCHAR2(15)
);

CREATE TABLE Branch (
    branch_id NUMBER PRIMARY KEY,
    branch_name VARCHAR2(50) NOT NULL,
    branch_city VARCHAR2(30)
);

CREATE TABLE Account (
    acc_no NUMBER PRIMARY KEY,
    cust_id NUMBER REFERENCES Customer(cust_id),
    branch_id NUMBER REFERENCES Branch(branch_id),
    balance NUMBER(12,2)
);

CREATE TABLE Loan (
    loan_no NUMBER PRIMARY KEY,
    cust_id NUMBER REFERENCES Customer(cust_id),
    branch_id NUMBER REFERENCES Branch(branch_id),
    amount NUMBER(12,2)
);
```

2. Insert 10 records into each table

```
INSERT INTO Customer VALUES (1,'Pranav','Pune','9876543210');
INSERT INTO Customer VALUES (2,'Amit','Mumbai','9876543211');
INSERT INTO Customer VALUES (3,'Priya','Pune',NULL);
INSERT INTO Customer VALUES (4,'Sneha','Nashik','9876543213');
INSERT INTO Customer VALUES (5,'Pooja','Mumbai','9876543214');
INSERT INTO Customer VALUES (6,'Rahul','Pune','9876543215');
INSERT INTO Customer VALUES (7,'Kiran','Satara',NULL);
INSERT INTO Customer VALUES (8,'Anjali','Mumbai','9876543217');
INSERT INTO Customer VALUES (9,'Rohit','Nashik','9876543218');
INSERT INTO Customer VALUES (10,'Payal','Pune','9876543219');

INSERT INTO Branch VALUES (101,'Akurdi','Pune');
INSERT INTO Branch VALUES (102,'Chinchwad','Pune');
INSERT INTO Branch VALUES (103,'Shivajinagar','Pune');
INSERT INTO Branch VALUES (104,'Andheri','Mumbai');
INSERT INTO Branch VALUES (105,'Dadar','Mumbai');
INSERT INTO Branch VALUES (106,'Nashik Road','Nashik');
INSERT INTO Branch VALUES (107,'Satara','Satara');
INSERT INTO Branch VALUES (108,'Kothrud','Pune');
INSERT INTO Branch VALUES (109,'Thane','Mumbai');
INSERT INTO Branch VALUES (110,'Wakad','Pune');
```

```
INSERT INTO Account VALUES (2001,1,101,50000);
INSERT INTO Account VALUES (2002,2,104,65000);
INSERT INTO Account VALUES (2003,3,102,35000);
INSERT INTO Account VALUES (2004,4,106,45000);
INSERT INTO Account VALUES (2005,5,105,80000);
INSERT INTO Account VALUES (2006,6,103,42000);
INSERT INTO Account VALUES (2007,7,107,25000);
INSERT INTO Account VALUES (2008,8,109,90000);
INSERT INTO Account VALUES (2009,9,106,55000);
INSERT INTO Account VALUES (2010,10,110,60000);

INSERT INTO Loan VALUES (3001,1,101,150000);
INSERT INTO Loan VALUES (3002,2,104,200000);
INSERT INTO Loan VALUES (3003,3,102,100000);
INSERT INTO Loan VALUES (3004,4,106,250000);
INSERT INTO Loan VALUES (3005,5,105,180000);
INSERT INTO Loan VALUES (3006,6,103,120000);
INSERT INTO Loan VALUES (3007,7,107,80000);
INSERT INTO Loan VALUES (3008,8,109,300000);
INSERT INTO Loan VALUES (3009,9,106,220000);
INSERT INTO Loan VALUES (3010,10,110,160000);

COMMIT;
```

3. SQL queries

1. Display all customers in alphabetical order.

```
SELECT *
FROM Customer
ORDER BY cust_name ASC;
```

2. Display customers whose names start with P.

```
SELECT cust_name, city
FROM Customer
WHERE cust_name LIKE 'P%';
```

3. Customers living in Pune or Mumbai whose phone is not NULL.

```
SELECT *
FROM Customer
WHERE city IN ('Pune', 'Mumbai')
AND phone IS NOT NULL;
```

4. Customers whose phone number is NULL.

```
SELECT *
FROM Customer
WHERE phone IS NULL;
```

5. Average account balance for each branch.

```
SELECT b.branch_name, AVG(a.balance) AS avg_balance
FROM Branch b
JOIN Account a ON b.branch_id = a.branch_id
GROUP BY b.branch_id, b.branch_name;
```

6. Branches with average account balance greater than ₹40,000.

```
SELECT b.branch_name, AVG(a.balance) AS avg_balance
FROM Branch b
JOIN Account a ON b.branch_id = a.branch_id
GROUP BY b.branch_id, b.branch_name
HAVING AVG(a.balance) > 40000;
```

7. Account balance after adding 10% interest, without changing the stored balance.

```
SELECT acc_no, balance,
       balance * 1.10 AS balance_with_interest
FROM Account;
```

8. Update a customer's phone number using their name.

```
UPDATE Customer
SET phone = '9999999999'
WHERE cust_name = 'Pranav';

COMMIT;
```

9. Delete customers who live in Pune.

```
DELETE FROM Customer
WHERE city = 'Pune';

COMMIT;
```

Run this query after completing the other queries, because deleting customers may conflict with account and loan foreign keys.

10. Display customers who have an account or a loan using UNION.

```
SELECT c.cust_id, c.cust_name
FROM Customer c
JOIN Account a ON c.cust_id = a.cust_id

UNION

SELECT c.cust_id, c.cust_name
FROM Customer c
JOIN Loan l ON c.cust_id = l.cust_id;
```

Experiment 2: Students and Enrolments

Aim: To perform joins, subqueries, views, and indexes.

1. Create tables and insert 10 records

```
CREATE TABLE Students (
    student_id NUMBER PRIMARY KEY,
    name VARCHAR2(50) NOT NULL,
    city VARCHAR2(30)
);

CREATE TABLE Enrolments (
    enroll_id NUMBER PRIMARY KEY,
    student_id NUMBER REFERENCES Students(student_id),
    course VARCHAR2(30),
    marks NUMBER(5,2)
);

INSERT INTO Students VALUES (1,'Amit','Pune');
INSERT INTO Students VALUES (2,'Priya','Mumbai');
INSERT INTO Students VALUES (3,'Rahul','Pune');
INSERT INTO Students VALUES (4,'Sneha','Nashik');
INSERT INTO Students VALUES (5,'Pooja','Mumbai');
INSERT INTO Students VALUES (6,'Karan','Pune');
INSERT INTO Students VALUES (7,'Neha','Satara');
INSERT INTO Students VALUES (8,'Rohan','Nashik');
INSERT INTO Students VALUES (9,'Anjali','Pune');
INSERT INTO Students VALUES (10,'Vikas','Mumbai');

INSERT INTO Enrolments VALUES (101,1,'DBMS',85);
INSERT INTO Enrolments VALUES (102,2,'Java',78);
INSERT INTO Enrolments VALUES (103,3,'DBMS',92);
INSERT INTO Enrolments VALUES (104,4,'Python',67);
INSERT INTO Enrolments VALUES (105,5,'Java',88);
INSERT INTO Enrolments VALUES (106,6,'DBMS',73);
INSERT INTO Enrolments VALUES (107,7,'Python',81);
INSERT INTO Enrolments VALUES (108,8,'DBMS',56);
INSERT INTO Enrolments VALUES (109,9,'Java',95);
INSERT INTO Enrolments VALUES (110,1,'Python',90);

COMMIT;
```

2. Queries

1. Display student names, courses, and marks.

```
SELECT s.name, e.course, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id;
```

2. Display all students, including those not enrolled in any course.

```
SELECT s.name, e.course, e.marks
FROM Students s
LEFT JOIN Enrolments e ON s.student_id = e.student_id;
```

3. Display all enrolments with student names.

```
SELECT e.*, s.name
FROM Enrolments e
LEFT JOIN Students s ON e.student_id = s.student_id;
```

4. Display all possible combinations of students and courses.

```
SELECT s.name, e.course
FROM Students s
CROSS JOIN Enrolments e;
```

5. Students whose marks are greater than the average marks.

```
SELECT DISTINCT s.student_id, s.name, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.marks > (SELECT AVG(marks) FROM Enrolments);
```

6. Display students who obtained marks greater than 80.

```
SELECT DISTINCT s.name, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.marks > 80;
```

7. Display students enrolled in DBMS.

```
SELECT DISTINCT s.name
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.course = 'DBMS';
```

8. Display student ID(s) with the highest marks.

```
SELECT DISTINCT student_id
FROM Enrolments
WHERE marks = (SELECT MAX(marks) FROM Enrolments);
```

9. Create a view and update marks through it.

```
CREATE VIEW student_course_view AS
SELECT s.name, e.course, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id;

SELECT * FROM student_course_view;

UPDATE student_course_view
SET marks = 89
WHERE name = 'Amit' AND course = 'DBMS';

COMMIT;
```

10. Create indexes.

```
CREATE INDEX idx_student_name ON Students(name);

CREATE UNIQUE INDEX idx_enroll_id
ON Enrolments(enroll_id);
```

Experiment 3: Student Course Queries and Views

Aim: To perform SQL joins, subqueries, aggregate functions, and index operations.

Use the `Students` and `Enrolments` tables from Experiment 2.

1. Display student names and courses.

```
SELECT s.name, e.course
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id;
```

2. Display all students and course details.

```
SELECT s.name, e.course, e.marks
FROM Students s
LEFT JOIN Enrolments e ON s.student_id = e.student_id;
```

3. Display DBMS students with their marks.

```
SELECT s.name, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.course = 'DBMS';
```

4. Display enrolments without a matching student.

```
SELECT e.*
FROM Enrolments e
LEFT JOIN Students s ON e.student_id = s.student_id
WHERE s.student_id IS NULL;
```

With an enforced foreign key, this query normally returns no rows.

5. Students who scored below the average marks.

```
SELECT s.name, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.marks < (SELECT AVG(marks) FROM Enrolments);
```

6. Students who obtained the highest marks.

```
SELECT s.name, e.marks
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM Enrolments);
```

7. Students belonging to Pune.

```
SELECT *
FROM Students
WHERE city = 'Pune';
```

8. Students enrolled in Python.

```
SELECT s.name
FROM Students s
JOIN Enrolments e ON s.student_id = e.student_id
WHERE e.course = 'Python';
```

9. Create a view showing average marks per course.

```
CREATE VIEW course_average_view AS
SELECT course, AVG(marks) AS average_marks
FROM Enrolments
GROUP BY course;

SELECT * FROM course_average_view;
```

10. Create indexes.

```
CREATE INDEX idx_students_city ON Students(city);

CREATE UNIQUE INDEX idx_students_id
ON Students(student_id);
```

Experiment 4: Student Grade Classification Using a Stored Procedure

Aim: To classify students according to their examination marks using a PL/SQL stored procedure.

1. Create tables

```
CREATE TABLE Stud_Marks (
    rollno NUMBER PRIMARY KEY,
    name VARCHAR2(50),
    total_marks NUMBER
);

CREATE TABLE Result (
    Roll NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    Class VARCHAR2(30)
);
```

2. Insert 10 records

```
INSERT INTO Stud_Marks VALUES (1,'Amit',1400);
INSERT INTO Stud_Marks VALUES (2,'Priya',1200);
INSERT INTO Stud_Marks VALUES (3,'Rahul',1000);
INSERT INTO Stud_Marks VALUES (4,'Sneha',980);
INSERT INTO Stud_Marks VALUES (5,'Pooja',920);
```

```
INSERT INTO Stud_Marks VALUES (6,'Karan',890);
INSERT INTO Stud_Marks VALUES (7,'Neha',850);
INSERT INTO Stud_Marks VALUES (8,'Rohan',830);
INSERT INTO Stud_Marks VALUES (9,'Anjali',800);
INSERT INTO Stud_Marks VALUES (10,'Vikas',700);

COMMIT;
```

3. Create stored procedure `proc_Grade`

```
CREATE OR REPLACE PROCEDURE proc_Grade AS
BEGIN
    DELETE FROM Result;

    FOR r IN (SELECT rollno, name, total_marks FROM Stud_Marks)
    LOOP
        IF r.total_marks BETWEEN 990 AND 1500 THEN
            INSERT INTO Result
            VALUES (r.rollno, r.name, 'Distinction');

        ELSIF r.total_marks BETWEEN 900 AND 989 THEN
            INSERT INTO Result
            VALUES (r.rollno, r.name, 'First Class');

        ELSIF r.total_marks BETWEEN 825 AND 899 THEN
            INSERT INTO Result
            VALUES (r.rollno, r.name, 'Higher Second Class');

        ELSE
            INSERT INTO Result
            VALUES (r.rollno, r.name, 'Below Higher Second Class');
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

4. Execute the procedure

```
EXEC proc_Grade;

SELECT * FROM Result;
```

Experiment 5: Calculate the Area of a Circle Using PL/SQL

Aim: To calculate the area of circles for radii from 5 to 9 and store the results in a table.

1. Create the table

```
CREATE TABLE areas (
    radius NUMBER PRIMARY KEY,
    area NUMBER(10,2)
);
```

2. PL/SQL program

```
DECLARE
    r NUMBER;
    calculated_area NUMBER;
BEGIN
    FOR r IN 5..9 LOOP
        calculated_area := 3.14159 * POWER(r, 2);

        INSERT INTO areas(radius, area)
        VALUES (r, calculated_area);
    END LOOP;

    COMMIT;
END;
/
```

3. Display the results

```
SELECT * FROM areas ORDER BY radius;
```

Expected output, rounded to two decimal places:

Radius	Area
5	78.54
6	113.10
7	153.94
8	201.06
9	254.47

Experiment 6: Cursor to Display Employees with Salary Greater Than ₹30,000

Aim: To use an explicit cursor in PL/SQL to fetch employee records one by one.

1. Create the Employee table

```
CREATE TABLE Employee (
    Emp_ID NUMBER PRIMARY KEY,
    Emp_Name VARCHAR2(50),
    Department VARCHAR2(30),
    Salary NUMBER(10,2)
);
```

2. Insert sample records

```
INSERT INTO Employee VALUES (1,'Amit','IT',45000);
INSERT INTO Employee VALUES (2,'Priya','HR',28000);
INSERT INTO Employee VALUES (3,'Rahul','IT',50000);
INSERT INTO Employee VALUES (4,'Sneha','Finance',35000);
INSERT INTO Employee VALUES (5,'Pooja','IT',32000);
INSERT INTO Employee VALUES (6,'Karan','HR',25000);
INSERT INTO Employee VALUES (7,'Neha','Finance',42000);
INSERT INTO Employee VALUES (8,'Rohan','IT',29000);
INSERT INTO Employee VALUES (9,'Anjali','HR',38000);
INSERT INTO Employee VALUES (10,'Vikas','Finance',31000);

COMMIT;
```

3. PL/SQL cursor program

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR emp_cursor IS
        SELECT Emp_ID, Emp_Name, Department, Salary
        FROM Employee
        WHERE Salary > 30000;

    v_id Employee.Emp_ID%TYPE;
    v_name Employee.Emp_Name%TYPE;
    v_dept Employee.Department%TYPE;
    v_salary Employee.Salary%TYPE;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor
        INTO v_id, v_name, v_dept, v_salary;

        EXIT WHEN emp_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || v_id ||
            ', Name: ' || v_name ||
            ', Department: ' || v_dept ||
```

```
            ', Salary: ' || v_salary
        );
    END LOOP;

    CLOSE emp_cursor;
END;
/
```

Experiment 7: Trigger for Library Audit

Aim: To create a trigger that stores the old values of library records whenever they are updated or deleted.

1. Create tables

```
CREATE TABLE Library (
    book_id NUMBER PRIMARY KEY,
    book_name VARCHAR2(100),
    author VARCHAR2(50),
    category VARCHAR2(30),
    date DATE,
    operation VARCHAR2(20)
);

CREATE TABLE Library_Audit (
    book_id NUMBER,
    book_name VARCHAR2(100),
    author VARCHAR2(50),
    category VARCHAR2(30),
    date DATE,
    operation VARCHAR2(20)
);
```

2. Insert 10 records

```
INSERT INTO Library VALUES
(1,'DBMS','Korth','Database',DATE '2025-01-01','INSERT');

INSERT INTO Library VALUES
(2,'Operating System','Galvin','Computer',DATE '2025-01-02','INSERT');

INSERT INTO Library VALUES
(3,'Computer Networks','Tanenbaum','Networking',DATE '2025-01-
03','INSERT');

INSERT INTO Library VALUES
(4,'Java Programming','Herbert Schildt','Programming',DATE '2025-01-
04','INSERT');

INSERT INTO Library VALUES
(5,'Python Basics','Guido','Programming',DATE '2025-01-05','INSERT');

INSERT INTO Library VALUES
(6,'Data Structures','Lipschutz','Computer',DATE '2025-01-06','INSERT');
```

```
INSERT INTO Library VALUES
(7,'C Programming','Dennis Ritchie','Programming',DATE '2025-01-
07','INSERT');

INSERT INTO Library VALUES
(8,'Artificial Intelligence','Russell','AI',DATE '2025-01-08','INSERT');

INSERT INTO Library VALUES
(9,'Software Engineering','Pressman','Software',DATE '2025-01-
09','INSERT');

INSERT INTO Library VALUES
(10,'Web Technology','Powell','Web',DATE '2025-01-10','INSERT');

COMMIT;
```

3. Create the trigger

```
CREATE OR REPLACE TRIGGER library_audit_trigger
BEFORE UPDATE OR DELETE ON Library
FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit
    (book_id, book_name, author, category, date, operation)
    VALUES
    (:OLD.book_id, :OLD.book_name, :OLD.author,
     :OLD.category, :OLD.date,
     CASE
         WHEN UPDATING THEN 'UPDATE'
         WHEN DELETING THEN 'DELETE'
     END);
END;
/
```

4. Test the trigger

```
UPDATE Library
SET category = 'Database Systems'
WHERE book_id = 1;

DELETE FROM Library
WHERE book_id = 2;

SELECT * FROM Library_Audit;
```

Experiment 8: Database Connectivity Using PHP and MySQL

Aim: To implement database connectivity with a front-end language and perform add, delete, edit, and display operations.

1. Create the database and table

Run in MySQL or phpMyAdmin:

```
CREATE DATABASE dbms_lab;
USE dbms_lab;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    city VARCHAR(50)
);
```

2. PHP program (`index.php`)

Save this file in your XAMPP `htdocs/dbms_lab/` directory.

```
<?php
$conn = new mysqli("localhost", "root", "", "dbms_lab");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (isset($_POST['add'])) {
    $name = $_POST['name'];
    $city = $_POST['city'];

    $stmt = $conn->prepare(
        "INSERT INTO students (name, city) VALUES (?, ?)"
    );
    $stmt->bind_param("ss", $name, $city);
    $stmt->execute();
    $stmt->close();
}

if (isset($_POST['delete'])) {
    $id = (int)$_POST['id'];

    $stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $stmt->close();
}

if (isset($_POST['edit'])) {
    $id = (int)$_POST['id'];
    $name = $_POST['name'];
    $city = $_POST['city'];

    $stmt = $conn->prepare(
        "UPDATE students SET name = ?, city = ? WHERE id = ?"
    );
    $stmt->bind_param("ssi", $name, $city, $id);
    $stmt->execute();
    $stmt->close();
}
?>

<!DOCTYPE html>
<html>
```

```
<head>
    <title>Student Management</title>
</head>
<body>
    <h2>Student Management System</h2>

    <form method="post">
        ID (for edit/delete):
        <input type="number" name="id"><br><br>

        Name:
        <input type="text" name="name"><br><br>

        City:
        <input type="text" name="city"><br><br>

        <button name="add">Add</button>
        <button name="edit">Edit</button>
        <button name="delete">Delete</button>
    </form>

    <h3>Student Records</h3>

    <?php
    $result = $conn->query("SELECT * FROM students");

    while ($row = $result->fetch_assoc()) {
        echo htmlspecialchars($row['id']) . " - " .
             htmlspecialchars($row['name']) . " - " .
             htmlspecialchars($row['city']) . "<br>";
    }

    $conn->close();
    ?>
</body>
</html>
```

3. Execution steps

1. Start Apache and MySQL from XAMPP.
2. Create the database and table in phpMyAdmin.
3. Save the code as `index.php` inside `htdocs/dbms_lab`.
4. Open `http://localhost/dbms_lab/index.php`.
5. Test the Add, Edit, and Delete operations.

Experiment 9: MongoDB Database Connectivity

Aim: To connect a front-end application to MongoDB and perform add, edit, delete, and display operations.

This example uses Node.js, Express, and the MongoDB Node.js driver.

1. Install dependencies

```
mkdir mongodb_lab
cd mongodb_lab
```

```
npm init -y
npm install express mongodb
```

2. Create `server.js`

```
const express = require("express");
const { MongoClient, ObjectId } = require("mongodb");

const app = express();
app.use(express.json());

const client = new MongoClient("mongodb://127.0.0.1:27017");

let students;

async function start() {
    await client.connect();

    const db = client.db("StudentDB");
    students = db.collection("students");

    app.get("/students", async (req, res) => {
        res.json(await students.find().toArray());
    });

    app.post("/students", async (req, res) => {
        const result = await students.insertOne(req.body);
        res.json(result);
    });

    app.put("/students/:id", async (req, res) => {
        const result = await students.updateOne(
            { _id: new ObjectId(req.params.id) },
            { $set: req.body }
        );
        res.json(result);
    });

    app.delete("/students/:id", async (req, res) => {
        const result = await students.deleteOne({
            _id: new ObjectId(req.params.id)
        });
        res.json(result);
    });

    app.listen(3000, () => {
        console.log("Server running on port 3000");
    });
}

start().catch(console.error);
```

3. Run the program

```
node server.js
```

Use a REST client such as Postman to test:

- `GET /students` — display all records.
- `POST /students` — add a student; send JSON such as `{"name":"Amit","city":"Pune"}`.
- `PUT /students/{id}` — update a student using their MongoDB ID.
- `DELETE /students/{id}` — delete a student using their MongoDB ID.

Result: The application performs CRUD operations through MongoDB connectivity.

Experiment 10: Bank Database Queries

Aim: To create a banking database and execute SQL queries using joins and aggregate functions.

1. Create tables

```
CREATE TABLE Branch (
    branch_name VARCHAR2(50) PRIMARY KEY,
    branch_city VARCHAR2(50),
    assets NUMBER(14,2)
);

CREATE TABLE Customer (
    cust_name VARCHAR2(50) PRIMARY KEY,
    cust_street VARCHAR2(100),
    cust_city VARCHAR2(50)
);

CREATE TABLE Account (
    acc_no NUMBER PRIMARY KEY,
    branch_name VARCHAR2(50)
        REFERENCES Branch(branch_name),
    balance NUMBER(12,2)
);

CREATE TABLE Depositor (
    cust_name VARCHAR2(50) REFERENCES Customer(cust_name),
    acc_no NUMBER REFERENCES Account(acc_no),
    PRIMARY KEY (cust_name, acc_no)
);

CREATE TABLE Loan (
    loan_no NUMBER PRIMARY KEY,
    branch_name VARCHAR2(50)
        REFERENCES Branch(branch_name),
    amount NUMBER(12,2)
);

CREATE TABLE Borrower (
    cust_name VARCHAR2(50) REFERENCES Customer(cust_name),
    loan_no NUMBER REFERENCES Loan(loan_no),
    PRIMARY KEY (cust_name, loan_no)
);
```

2. Insert suitable sample data

```
INSERT INTO Branch VALUES ('Akurdi','Pune',5000000);
```

```
INSERT INTO Branch VALUES ('Chinchwad','Pune',6000000);
INSERT INTO Branch VALUES ('Shivajinagar','Pune',7000000);
INSERT INTO Branch VALUES ('Andheri','Mumbai',8000000);
INSERT INTO Branch VALUES ('Dadar','Mumbai',6500000);

INSERT INTO Customer VALUES ('Amit','MG Road','Pune');
INSERT INTO Customer VALUES ('Priya','FC Road','Pune');
INSERT INTO Customer VALUES ('Rahul','Main Road','Mumbai');
INSERT INTO Customer VALUES ('Sneha','College Road','Nashik');
INSERT INTO Customer VALUES ('Pooja','Station Road','Mumbai');

INSERT INTO Account VALUES (2001,'Akurdi',50000);
INSERT INTO Account VALUES (2002,'Chinchwad',65000);
INSERT INTO Account VALUES (2003,'Shivajinagar',40000);
INSERT INTO Account VALUES (2004,'Andheri',80000);
INSERT INTO Account VALUES (2005,'Dadar',55000);

INSERT INTO Depositor VALUES ('Amit',2001);
INSERT INTO Depositor VALUES ('Priya',2002);
INSERT INTO Depositor VALUES ('Rahul',2003);
INSERT INTO Depositor VALUES ('Sneha',2004);
INSERT INTO Depositor VALUES ('Pooja',2005);

INSERT INTO Loan VALUES (3001,'Akurdi',200000);
INSERT INTO Loan VALUES (3002,'Chinchwad',350000);
INSERT INTO Loan VALUES (3003,'Shivajinagar',450000);
INSERT INTO Loan VALUES (3004,'Andheri',600000);
INSERT INTO Loan VALUES (3005,'Dadar',550000);

INSERT INTO Borrower VALUES ('Amit',3001);
INSERT INTO Borrower VALUES ('Priya',3002);
INSERT INTO Borrower VALUES ('Rahul',3003);
INSERT INTO Borrower VALUES ('Sneha',3004);
INSERT INTO Borrower VALUES ('Pooja',3005);

COMMIT;
```

3. Queries

1. Create all tables with constraints: Completed above.

2. Add a phone number to Customer.

```
ALTER TABLE Customer ADD phone VARCHAR2(15);
```

3. Delete the phone number attribute.

```
ALTER TABLE Customer DROP COLUMN phone;
```

Run query 3 only after completing any queries that need the phone column.

4. Find all branch names in the Loan relation.

```
SELECT DISTINCT branch_name
FROM Loan;
```

5. Find customers who have loans, with loan numbers and amounts.

```
SELECT c.cust_name, l.loan_no, l.amount
FROM Customer c
JOIN Borrower b ON c.cust_name = b.cust_name
JOIN Loan l ON b.loan_no = l.loan_no;
```

6. Customers who have a loan from Akurdi, in alphabetical order.

```
SELECT DISTINCT c.cust_name
FROM Customer c
JOIN Borrower b ON c.cust_name = b.cust_name
JOIN Loan l ON b.loan_no = l.loan_no
WHERE l.branch_name = 'Akurdi'
ORDER BY c.cust_name;
```

7. Average account balance at Chinchwad.

```
SELECT AVG(balance) AS average_balance
FROM Account
WHERE branch_name = 'Chinchwad';
```

8. Number of depositors at each branch.

```
SELECT a.branch_name, COUNT(DISTINCT d.cust_name) AS depositors
FROM Account a
JOIN Depositor d ON a.acc_no = d.acc_no
GROUP BY a.branch_name;
```

9. Total loan amount for each branch.

```
SELECT branch_name, SUM(amount) AS total_loan
FROM Loan
GROUP BY branch_name;
```

10. Branches where average loan amount exceeds ₹5,00,000.

```
SELECT branch_name, AVG(amount) AS average_loan
FROM Loan
GROUP BY branch_name
HAVING AVG(amount) > 500000;
```

Experiment 11: Banking Database — Alter, Aggregate and Filtering

Use the banking tables from Experiment 10.

1. Rename the `assets` attribute to `Property`.

```
ALTER TABLE Branch RENAME COLUMN assets TO Property;
```

2. Find loans at Akurdi with amounts greater than ₹12,000.

```
SELECT loan_no
FROM Loan
WHERE branch_name = 'Akurdi'
```

```
AND amount > 12000;
```

3. Average account balance at each branch.

```
SELECT branch_name, AVG(balance) AS average_balance
FROM Account
GROUP BY branch_name;
```

4. Branches with average account balance greater than ₹12,000.

```
SELECT branch_name, AVG(balance) AS average_balance
FROM Account
GROUP BY branch_name
HAVING AVG(balance) > 12000;
```

5. Count the customer records.

```
SELECT COUNT(*) AS total_customers
FROM Customer;
```

6. Calculate the total loan amount.

```
SELECT SUM(amount) AS total_loan_amount
FROM Loan;
```

7. Delete loans with amounts between ₹1,300 and ₹1,500.

```
DELETE FROM Loan
WHERE amount BETWEEN 1300 AND 1500;

COMMIT;
```

8. Find customers whose street address is unavailable.

```
SELECT *
FROM Customer
WHERE cust_street IS NULL;
```

9. Find branches whose city is available.

```
SELECT *
FROM Branch
WHERE branch_city IS NOT NULL;
```

10. Find customers whose names start with A.

```
SELECT *
FROM Customer
WHERE cust_name LIKE 'A%';
```

Experiment 12: MongoDB Employee Collection

Aim: To create an `ICEM` database and execute MongoDB queries on employee records.

1. Create database and insert records

Run in the MongoDB shell (`mongosh`).

```
use ICEM

db.employee.insertMany([
  {empid:1, ename:"Amit", dept:"Computer", salary:45000},
  {empid:2, ename:"Priya", dept:"IT", salary:35000},
  {empid:3, ename:"Rahul", dept:"Computer", salary:50000},
  {empid:4, ename:"Sneha", dept:"HR", salary:28000},
  {empid:5, ename:"Pooja", dept:"IT", salary:40000},
  {empid:6, ename:"Karan", dept:"Computer", salary:32000},
  {empid:7, ename:"Neha", dept:"HR", salary:25000},
  {empid:8, ename:"Rohan", dept:"IT", salary:42000},
  {empid:9, ename:"Anjali", dept:"Computer", salary:38000},
  {empid:10, ename:"Vikas", dept:"Finance", salary:30000}
])
```

2. Queries

1. Display all employees.

```
db.employee.find()
```

2. Employees working in Computer department.

```
db.employee.find({dept:"Computer"})
```

3. Employees working in Computer or IT department.

```
db.employee.find({
  $or: [{dept:"Computer"}, {dept:"IT"}]
})
```

4. Employees whose salary is greater than ₹30,000.

```
db.employee.find({salary:{$gt:30000}})
```

5. Employees earning at least ₹40,000 in Computer department.

```
db.employee.find({
  salary:{$gte:40000},
  dept:"Computer"
})
```

6. Create and drop an index on `ename`.

```
db.employee.createIndex({ename:1})
db.employee.dropIndex("ename_1")
```

7. Employees earning between ₹20,000 and ₹40,000, inclusive.

```
db.employee.find({
  salary:{$gte:20000, $lte:40000}
```

```
})
```

8. Employees whose department is not IT.

```
db.employee.find({dept:{$ne:"IT"}})
```

9. Total salary paid in each department.

```
db.employee.aggregate([
  {
    $group:{
      _id:"$dept",
      total_salary:{$sum:"$salary"}
    }
  }
])
```

10. Average salary in each department.

```
db.employee.aggregate([
  {
    $group:{
      _id:"$dept",
      average_salary:{$avg:"$salary"}
    }
  }
])
```

Experiment 13: MongoDB Student Collection

Aim: To create a `Student` database and perform queries on student records.

1. Create database and insert records

```
use Student
```

```
db.student.insertMany([
  {roll_no:1, sname:"Amit", dept:"Computer", marks:85},
  {roll_no:2, sname:"Priya", dept:"IT", marks:75},
  {roll_no:3, sname:"Rahul", dept:"Computer", marks:92},
  {roll_no:4, sname:"Sneha", dept:"HR", marks:65},
  {roll_no:5, sname:"Pooja", dept:"IT", marks:88},
  {roll_no:6, sname:"Karan", dept:"Computer", marks:35},
  {roll_no:7, sname:"Neha", dept:"HR", marks:78},
  {roll_no:8, sname:"Rohan", dept:"IT", marks:55},
  {roll_no:9, sname:"Anjali", dept:"Computer", marks:95},
  {roll_no:10, sname:"Vikas", dept:"Finance", marks:40}
])
```

2. Queries

1. Display all students.

```
db.student.find()
```

2. Students belonging to Computer department.

```
db.student.find({dept:"Computer"})
```

3. Students belonging to Computer or IT department.

```
db.student.find({
  $or: [{dept:"Computer"}, {dept:"IT"}]
})
```

4. Students whose marks are greater than 70.

```
db.student.find({marks:{$gt:70}})
```

5. Students scoring at least 80 in Computer department.

```
db.student.find({
  marks:{$gte:80},
  dept:"Computer"
})
```

6. Students whose marks are below 40 or who belong to IT.

```
db.student.find({
  $or: [
    {marks:{$lt:40}},
    {dept:"IT"}
  ]
})
```

7. Students scoring between 40 and 80, inclusive.

```
db.student.find({
  marks:{$gte:40, $lte:80}
})
```

8. Students whose department is not IT.

```
db.student.find({dept:{$ne:"IT"}})
```

9. Total marks in each department.

```
db.student.aggregate([
  {
    $group:{
      _id:"$dept",
      total_marks:{$sum:"$marks"}
    }
  }
])
```

10. Create and drop an index on `sname`.

```
db.student.createIndex({sname:1})
db.student.dropIndex("sname_1")
```

Experiment 14: Customer, Branch, Account and Loan — Advanced SQL

Aim: To perform SQL operations using `LIKE`, `GROUP BY`, `HAVING`, `UPDATE`, `DELETE`, `INTERSECT`, and `MINUS`.

Use the four tables created in Experiment 1.

1. Customers living in Pune or Mumbai, in alphabetical order

```
SELECT *
FROM Customer
WHERE city IN ('Pune', 'Mumbai')
ORDER BY cust_name ASC;
```

2. Customers whose names contain the letter 'a'

```
SELECT *
FROM Customer
WHERE LOWER(cust_name) LIKE '%a%';
```

3. Customers whose phone number is not NULL

```
SELECT *
FROM Customer
WHERE phone IS NOT NULL;
```

4. Customers who live in Pune and whose names start with P, or who live in Mumbai

```
SELECT *
FROM Customer
WHERE (city = 'Pune' AND cust_name LIKE 'P%')
   OR city = 'Mumbai';
```

5. Maximum and average loan amount for each branch

```
SELECT b.branch_name,
       MAX(l.amount) AS maximum_loan,
       AVG(l.amount) AS average_loan
FROM Branch b
JOIN Loan l ON b.branch_id = l.branch_id
GROUP BY b.branch_id, b.branch_name;
```

6. Branches whose total loan amount is greater than ₹1,00,000

```
SELECT b.branch_name,
       SUM(l.amount) AS total_loan
```

```
FROM Branch b
JOIN Loan l ON b.branch_id = l.branch_id
GROUP BY b.branch_id, b.branch_name
HAVING SUM(l.amount) > 100000;
```

7. Display account balances after adding ₹500

```
SELECT acc_no,
       balance,
       balance + 500 AS updated_display_balance
FROM Account;
```

This query only displays the calculated balance. It does not modify the stored value.

8. Change the branch of account number 2005

```
UPDATE Account
SET branch_id = 101
WHERE acc_no = 2005;

COMMIT;
```

Here, branch ID `101` represents the Akurdi branch in the sample data. Change it to another existing branch ID if required.

9. Delete customers whose phone number is NULL

```
DELETE FROM Customer
WHERE phone IS NULL;

COMMIT;
```

If foreign-key constraints prevent deletion, resolve the related account and loan records first. Run this query after completing the other experiments that require the original customer records.

10(a). Customers who have both an account and a loan using INTERSECT

```
SELECT cust_id
FROM Account

INTERSECT

SELECT cust_id
FROM Loan;
```

To display customer names as well:

```
SELECT cust_name
FROM Customer
WHERE cust_id IN (
    SELECT cust_id FROM Account
    INTERSECT
    SELECT cust_id FROM Loan
);
```

10(b). Customers who have an account but no loan using MINUS

```
SELECT cust_id
FROM Account

MINUS

SELECT cust_id
FROM Loan;
```

To display customer names:

```
SELECT cust_name
FROM Customer
WHERE cust_id IN (
    SELECT cust_id FROM Account
    MINUS
    SELECT cust_id FROM Loan
);
```